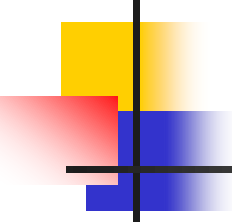


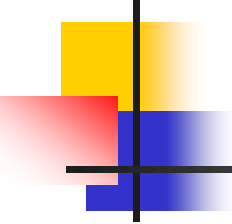
MÔ HÌNH MVC

MODEL VIEW CONTROLLER



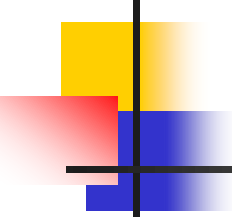
1. Giới thiệu

- Tại sao phải có mô hình MVC?
- Một công việc được giao cho nhiều người làm theo từng công đoạn khác nhau.
- Như vậy công việc đó có tốt đẹp và khả quan không? Phải gọi là thúc đẩy làm nhanh hơn là chẳng khác.
- Chưa kể đến việc hư hỏng xảy ra, bộ phận nào làm kém chất lượng ở công đoạn nào thì bộ phận đó làm lại công đoạn đó, không ảnh hưởng đến công việc của những bộ phận khác.



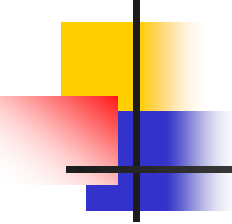
1. Giới thiệu

- Mô hình MVC cũng vậy đây
- Đơn giản là chia công việc cho các phần khác nhau trong một ứng dụng.
- Các phần thực hiện mỗi công việc khác nhau, chỉ làm và trả về kết quả.
- Có lỗi phần nào thì sửa code phần đó, không có việc mở cả project lên để tìm lỗi.
- Tóm lại, dễ dễ dàng sửa chữa, gỡ rối, bảo trì và phát triển ứng dụng



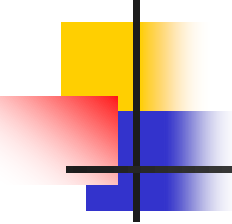
1. Giới thiệu

- Mô hình **MVC** được sử dụng lần đầu tiên trong Smalltalk, sau đó được sử dụng phổ biến trong ngôn ngữ lập trình Java.
- **Smalltalk** là ngôn ngữ lập trình hướng đối tượng, kiểu dữ liệu động và có tính phản xạ. **Smalltalk** được thiết kế nhằm bảo vệ tối đa các đặc điểm ngắn gọn và trong sáng của nó và không hướng đến tính khả dụng hay tính hiệu năng.
- Mô hình MVC hiện nay rất phổ biến trong các framework PHP



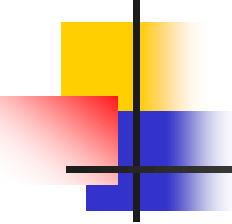
1. Giới thiệu

- Ý nghĩa của mô hình nhằm tách biệt hoàn toàn phần lưu trữ và xử lý dữ liệu (**Model**) với thành phần trình bày kết quả cho người dùng (**View**)
- Cho phép người lập trình có thể tách biệt công việc trong **quá trình xây dựng chức năng của ứng dụng** và **quá trình xây dựng giao diện** cho người dùng.



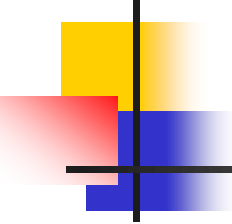
1. Giới thiệu

- Việc thay đổi thành phần của dữ liệu (model) sẽ không ảnh hưởng nhiều đến giao diện của người dùng
- Mô hình đưa ra Model để không cho người dùng thao tác trực tiếp vào dữ liệu vật lý (cơ sở dữ liệu hay là tập tin) mà phải thông qua Model
- Vì vậy cho dù dữ liệu vật lý thay đổi cấu trúc nhưng cấu trúc Model cho việc truy cập, xử lý, lưu trữ dữ liệu sẽ không bị ảnh hưởng



1. Giới thiệu

- Một user A muốn xem các phim chọn lọc được lựa chọn từ các đĩa CD/DVD
- Để đọc được các đĩa CD/DVD thì phải dùng đầu máy đọc đĩa
- Đầu máy này đặc biệt không có nút điều khiển trực tiếp, đầu máy này nhận hàng loạt các đĩa
- Do vậy muốn kích hoạt và chọn đĩa xem, người dùng A bắt buộc phải sử dụng remote control – điều khiển từ xa

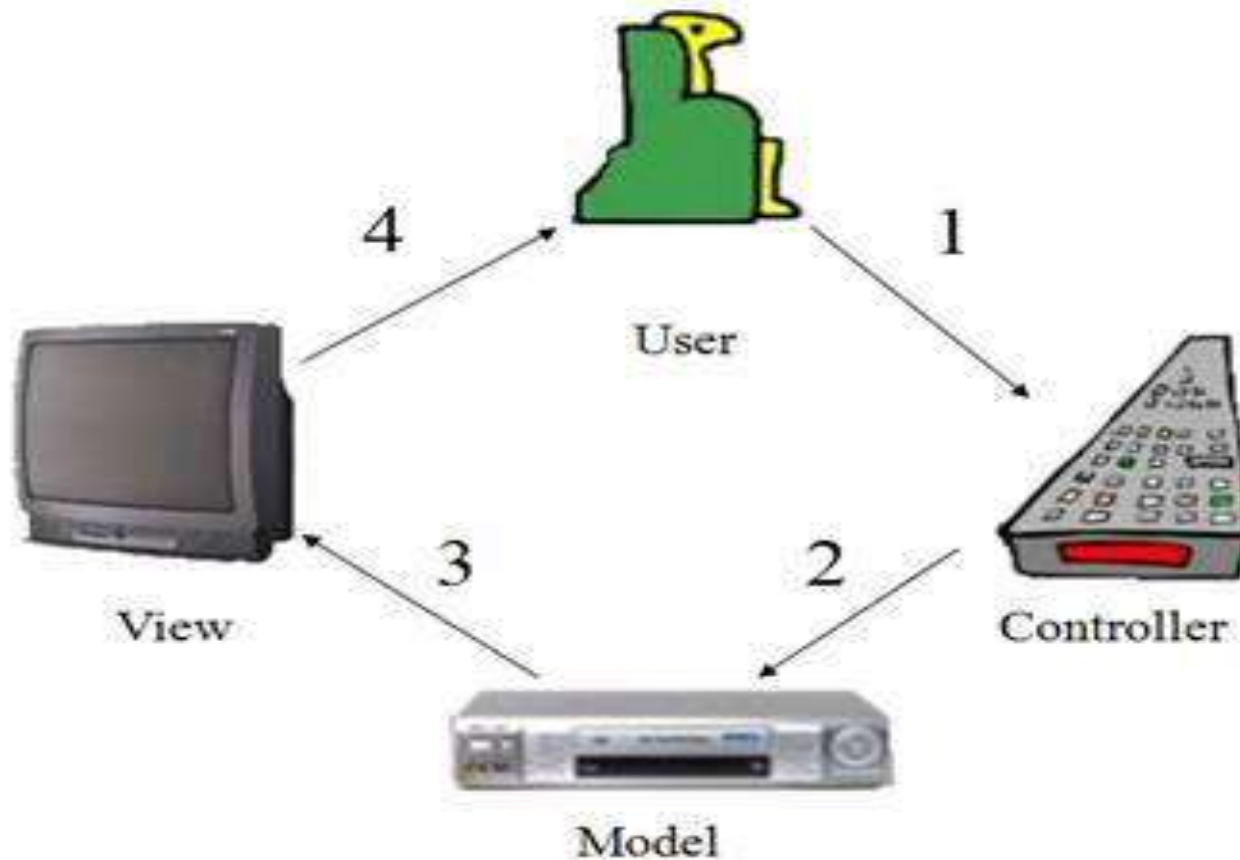


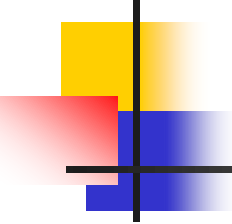
1. Giới thiệu

- Như vậy, chúng ta sẽ nhận thấy cách thức A xem phim như sau :
 - Người dùng A sẽ sử dụng remote control điều khiển đầu máy để chọn đĩa mà mình thích xem nhất để mở nó ra
 - Đĩa được chọn từ đầu máy được đọc và trình chiếu trên màn hình TV
 - Khi dữ liệu đã hiển thị trên màn hình TV thì người dùng A sẽ thấy được nội dung
 - Cách thức này sẽ được lặp đi lặp lại khi người dùng A chọn một phím khác, vẫn dùng bộ điều khiển để chọn đĩa và đầu đĩa sẽ đưa dữ liệu lên màn hình TV

1. Giới thiệu

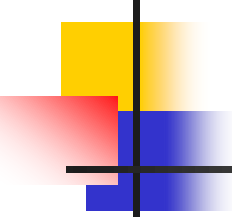
- Xét ví dụ như sau:





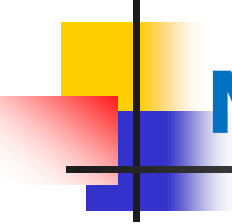
1. Giới thiệu

- Chúng ta nhận thấy rằng:
 - Đầu máy là nơi xử lý dữ liệu, chọn lựa cách thức xử lý, nội dung cần thiết – Model
 - Màn hình TV chỉ làm nhiệm vụ duy nhất để trình bày kết quả mà đầu máy – Model đã thực hiện, được lựa chọn. Màn hình TV không thể lựa chọn và không có cách chọn lựa là trình bày các thành phần truyền đến đã được xử lý. Màn hình TV là View
 - Thành phần hỗ trợ đưa dữ liệu từ Model đến View đó là bộ điều khiển, ngoài ra bộ điều khiển cũng là nơi kết nối người dùng với đầu máy với màn hình TV.
 - Chức năng của điều khiển là chọn đúng model để đưa ra view. Bộ điều khiển – remote control là một Controller



1. Giới thiệu

- Mô hình MVC là mô hình gồm 3 lớp: Model, View, Controller.
 - **Model:** Lớp này chịu trách nhiệm quản lí dữ liệu, giao tiếp với cơ sở dữ liệu, chịu trách nhiệm lưu trữ hoặc truy vấn dữ liệu.
 - **View:** Lớp này chính là giao diện của ứng dụng, chịu trách nhiệm biểu diễn dữ liệu của ứng dụng thành các dạng nhìn thấy được.
 - **Controller:** Lớp này đóng vai trò quản lí và điều phối luồng hoạt động của ứng dụng. Nhận request từ client, điều phối các Model và View để có thể cho ra output thích hợp và trả kết quả về cho người dùng.



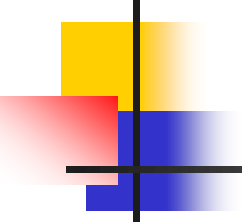
Mối Quan Hệ Giữa Các Thành Phần

- **Mối quan hệ giữa Controller và Model**

Ý tưởng đó là khi chúng ta thay đổi đầu máy đọc, bộ điều khiển từ xa có thay đổi theo hay không? Hay chúng ta vẫn dùng cái điều khiển cũ (không tính điều khiển đa năng)? Bộ điều khiển được thiết kế dựa trên đầu máy hay đầu máy được thiết kế dựa trên đồ điều khiển?

Đó là controller phụ thuộc vào model bởi vì controller được thiết kế để kết nối với model, điều khiển, truy xuất model.

Do vậy, model có sự thay đổi là controller phải thay đổi theo để có thể truy xuất và điều khiển model cho phù hợp



Mối Quan Hệ Giữa Các Thành Phần

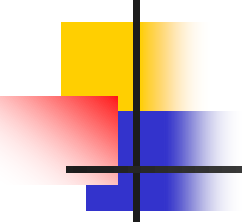
- **Mối quan hệ giữa Model và View**

View lệ thuộc vào Model bởi vì khi các tổ chức interface của Model thay đổi nghĩa là View phải thay đổi theo

Chúng ta tránh nhầm lẫn về khái niệm ở trên là tách biệt giữa View và Model là có lợi nhưng tại sao ở đây View lại lệ thuộc Model?

Khái niệm của chúng ta ở đây đó là tách biệt dữ liệu và thành phần xử lý bên trong, hệ thống sẽ trở nên uyển chuyển khi chúng ta thay đổi thành phần cài đặt trong Model, không phải thay đổi interface Model.

Và khái niệm trên đã nêu rõ giữa dữ liệu vật lý và dữ liệu được chứa trong Model



Mối Quan Hệ Giữa Các Thành Phần

- **Mối quan hệ giữa View và Controller**

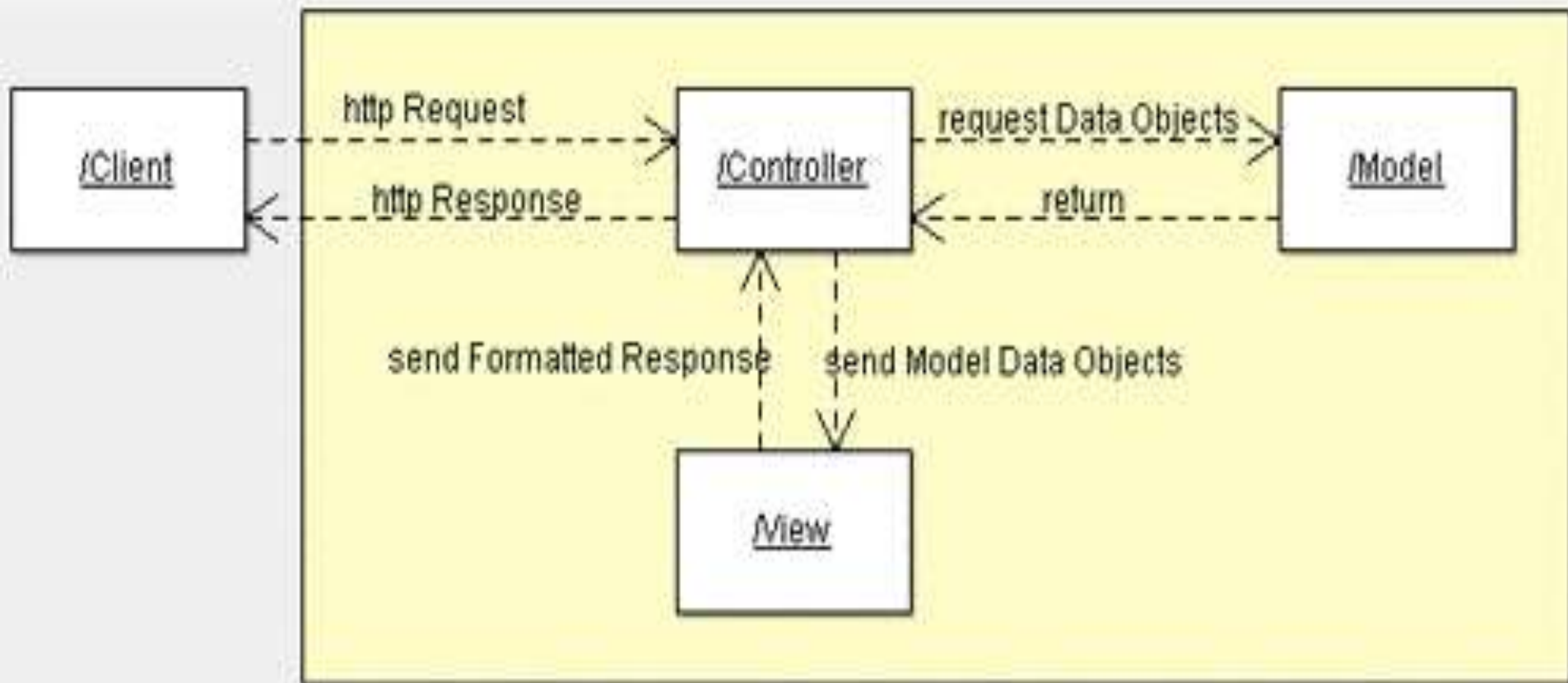
Controller sẽ là đối tượng lựa chọn View.

Tùy thuộc theo việc yêu cầu sử dụng của người dùng

Tùy thuộc theo kết quả xử lý của Model

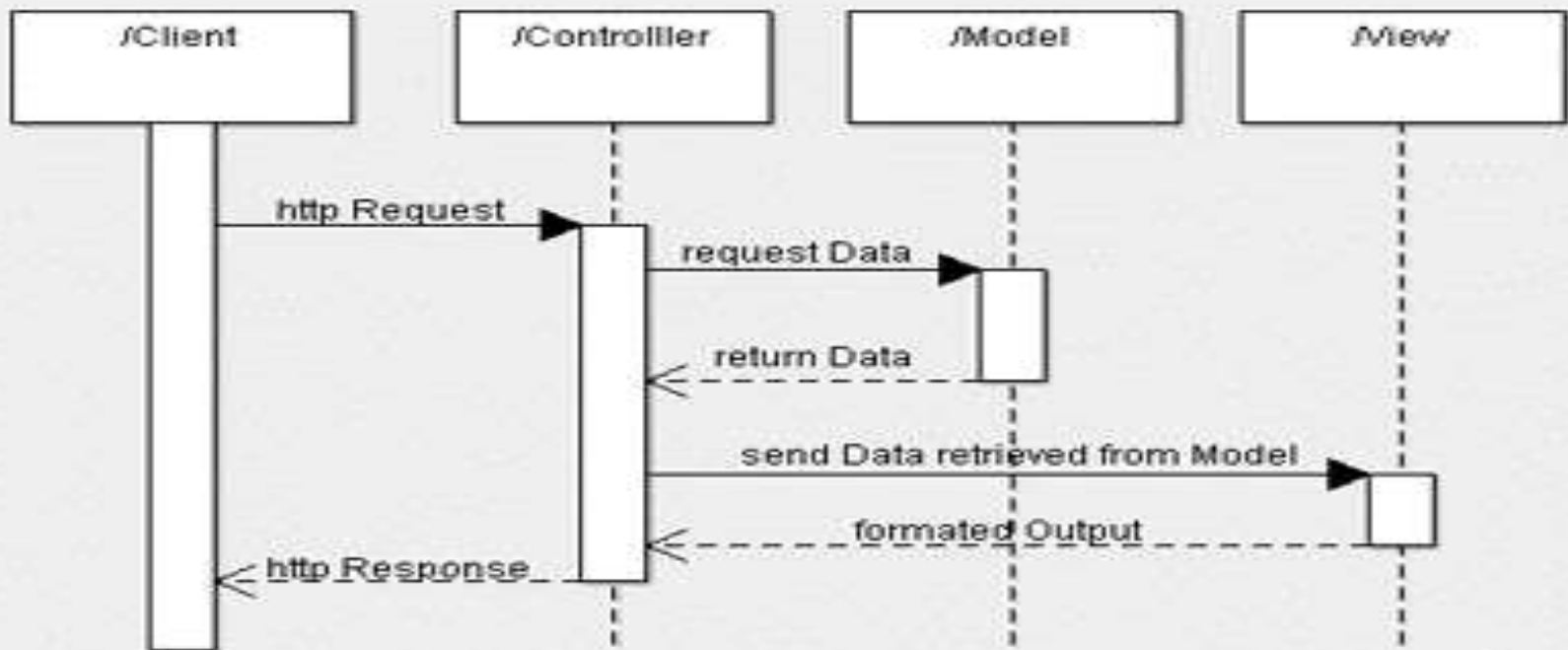
1. Giới thiệu

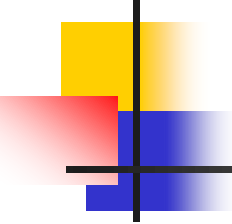
- Ta có thể mô tả lại hoạt động của mô hình MVC thông qua sơ đồ sau:



2. Ứng dụng minh họa

- Sơ đồ tuần tự mô tả luồng chạy của một xử lý trong mô hình MVC





2. Ứng dụng minh họa

- Ứng dụng minh họa có các chức năng sau:

Trang chủ: index.php

Các chức năng hệ thống:

Xem thông tin sinh viên

Chèn thông tin sinh viên

Sửa thông tin sinh viên

.....

Quản lý sinh viên: C_Student.php

Danh sách sinh viên

Sinh viên 1

Sinh viên 2

.....

Chi tiết sinh viên: C_Student.php

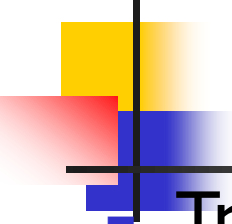
Chi tiết sinh viên:

Sinh viên 1:

Tên sinh viên

Tuổi sinh viên

Trường

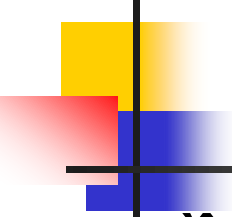


2. Ứng dụng minh họa

- Trong ví dụ sau đây, ta sẽ tổ chức source code minh họa như mô hình sau đây:

- **MVC-EXAMPLE:**

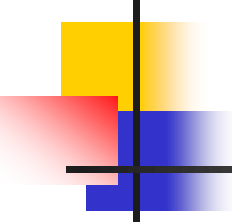
- **Controller**
 - **C_Student.php**
- **Model**
 - **M_Student.php**
 - **E_Student.php**
- **View**
 - **StudentList.htm**
 - **StudentDetail.htm**
 -
- **Index.php**



2. Ứng dụng minh họa

a. Tầng Controller

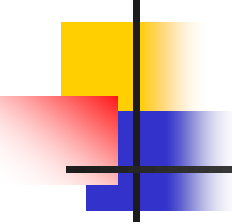
- Đây là tầng điều phối của ứng dụng: nhận request từ client, phân tích request, gọi tầng Model để lấy dữ liệu và tầng View để tổ chức hiển thị dữ liệu.
- Ở đây ta nói tới chức năng quản lý thông tin sinh viên, nó sẽ được điều phối bởi Controller: **C_Student.php**
- Trong chức năng có 2 chức năng con:
 - **Hiển thị danh sách sinh viên**
 - **Hiển thị chi tiết sinh viên**
- Vậy nên nó sẽ phân biệt từng tính năng bằng cách kiểm tra tham số đầu vào là "**stid**". Tùy trường hợp mà **Controller** sẽ gọi **View** tương ứng



2. Ứng dụng minh họa

■ a. Tầng Controller

```
1 <?php
2 include_once("../Model/M_Student.php");
3 class Ctrl_Student{
4     public function invoke(){
5         if(isset($_GET['stid'])){
6             $modelStudent = new Model_Student();
7             $student = $modelStudent->getStudentDetail($_GET['stid']);
8             include_once("../View/StudentDetail.html");
9         }
10        else{
11            $modelStudent = new Model_Student();
12            $studentList = $modelStudent->getAllStudent();
13            include_once("../View/StudentList.html");
14        }
15    }
16    $C_Student = new Ctrl_Student();
17    $C_Student->invoke();
18 ?>
```



2. Ứng dụng minh họa

b. Tầng Model và các lớp thực thể (Entity class)

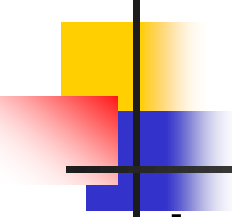
- Nhiệm vụ chính của tầng này là việc thao tác với cơ sở dữ liệu để lưu trữ cũng như truy vấn dữ liệu.
- Các thao tác với CSDL phải được đặt hết ở trong tầng này, nó sẽ chứa các thao tác cơ bản để truy vấn CSDL như: **xem, thêm, sửa, xoá,...**



2. Ứng dụng minh họa

b. Tầng Model và các lớp thực thể (Entity class)

- Trong ví dụ này, tầng Model bao gồm 2 class:
 - Model_Student
 - Entity_Student
- Ngoài ra thì tầng này còn chứa các lớp thực thể (Entity class), nó là các lớp để mô phỏng những đối tượng trong thực tế.
- Lớp **Entity_Student** chính là 1 thực thể mô phỏng về một student trong thực tế (tên, tuổi, trường, ...)
- Lớp **Model_Student** sẽ sử dụng lớp thực thể này để cấu trúc dữ liệu sẽ lấy từ Database. Lớp **Model_Student** đóng vai trò giao tiếp với CSDL.

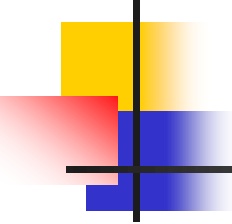


2. Ứng dụng minh họa

b. Tầng Model và các lớp thực thể (Entity class)

```
1 <?php
2 class Entity_Student
3 {
4     public $id;
5     public $name;
6     public $age;
7     public $university;
8
9     public function __construct($_id, $_name, $_age, $_university)
10    {
11        $this->id = $_id;
12        $this->name = $_name;
13        $this->age = $_age;
14        $this->university = $_university;
15    }
16 }
17 ?>
```

Entity class, mô tả các bảng trong CSDL

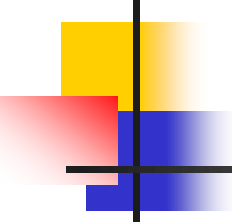


2. Ứng dụng minh họa

■ b. Tầng Model và các lớp thực thể (Entity class)

```
1 <?php
2 include_once("E_Student.php");
3 class Model_Student {
4     public function __construct() {}
5     public function getAllStudent(){
6         $link = mysqli_connect("localhost","root","") or die ("Khong the ket noi den CSDL MySQL");
7         mysqli_select_db($link,"DULIEU");
8         $sql = "select * from sinhvien";
9         $rs = mysqli_query($link, $sql);
10        $i=0;
11        while($row = mysqli_fetch_array($rs)){
12            $id = $row['id']; $name = $row['name'];
13            $age = $row['age']; $university = $row['university'];
14            while($i!=$id) $i++;
15            $students[$i++] = new Entity_Student($id, $name, $age, $university);
16        }
17        return $students;
18    }
19    public function getStudentDetail($stid){
20        // Load du lieu tu CSDL
21        $allStudent = $this->getAllStudent();
22        return $allStudent[$stid];
23    }
24 }
25 ?>
```

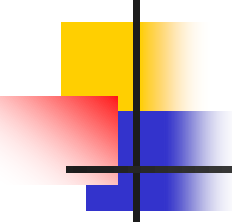
Lớp Model phụ trách phần liên kết dữ liệu Database



2. Ứng dụng minh họa

c. Tầng View

- Lớp này dùng để định dạng hiển thị dữ liệu lên cho người dùng, do ta có 2 tính năng trong phần “Quản lý thông tin sinh viên” là:
 - **Danh sách sinh viên**
 - **Chi tiết sinh viên**(nên ta sẽ có 2 View khác nhau tương ứng)
- **Controller** sẽ truyền các biến dữ liệu vào các **View** để chúng có thể dùng các dữ liệu này để hiển thị lên.
- Việc này được thực hiện bằng cách nhúng mã PHP vào trong mã HTML.

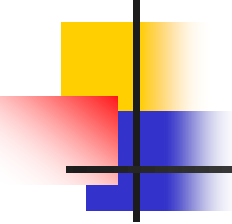


2. Ứng dụng minh họa

c. Tầng View

```
1 <html>|
2 <head>
3   <meta charset="UTF-8"/>
4   <title>Danh sách sinh viên</title>
5 </head>
6 <body>
7   <h2>Danh sách sinh viên:</h2>
8   <?php
9     for($i = 1; $i <= sizeof($studentList); $i++)
10     {
11       echo "<p>" . $i . ". <a href='?stdid=" . $studentList[$i]->id . "'>" . $studentList[$i]->name
12         . "</a></p>";
13     }
14   ?>
15   <p><a href=" ../index.php">Home page</a></p>
16 </body>
17 </html>
```

View của “Danh sách sinh viên”

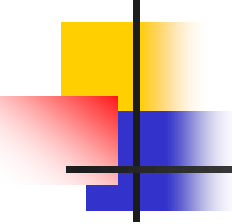


2. Ứng dụng minh họa

c. Tầng View

```
1 |<html>
2 |<head>
3 |   <meta charset="UTF-8"/>
4 |   <title>Thông tin chi tiết sinh viên</title>
5 |</head>
6 |
7 |<body>
8 |   <h2>Chi tiết sinh viên:</h2>
9 |   <?php
10 |     echo "<p><b>" . $student->name . "</b></p>";
11 |     echo "<p> Age: " . $student->age . "</p>";
12 |     echo "<p> University: " . $student->university . "</p><br>";
13 |   ?>
14 |   <p><a href="javascript:history.back()">Back</a></p>
15 | </body>
16 |</html>
```

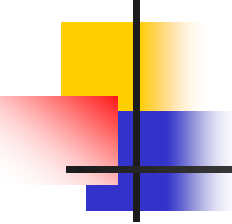
View của “Chi tiết sinh viên”



2. Ứng dụng minh họa

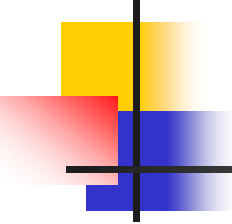
c. Tầng View

- Việc nhúng mã PHP vào HTML khiến mã nguồn trở nên phức tạp và khó kiểm soát, việc này dẫn tới sự ra đời của các “template engine”
- Các “template engine” tách biệt mã PHP ra khỏi mã HTML nên sẽ khiến code trở nên rõ ràng hơn rất nhiều.
- Đối với các ứng dụng thuần AJAX, phần View có thể được triển khai bởi JavaScript, sử dụng cấu trúc JSON



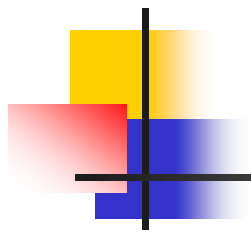
Một số ưu điểm của MVC :

- Model và View tách biệt, nên ứng dụng linh hoạt hơn, dễ bảo trì hơn các website truyền thống, một nùi code trong một file
- Model và View có thể coi là tách biệt, độc lập.
 - Có thể một máy chủ A chỉ làm nhiệm vụ xử lý Model, back-end
 - Một máy chủ B chỉ làm nhiệm vụ xử lý giao diện, front-end,...



Một số ưu điểm của MVC :

- Mô hình MVC là mô hình thông dụng và tiện lợi hàng đầu trong việc phát triển ứng dụng web
- Một vài lợi ích của MVC:
 - Tách biệt Model và View, tạo sự linh hoạt khi thay đổi ứng dụng (giao diện, xử lý, ...)
 - Tầng Model và View có thể thay đổi, chỉnh sửa một cách độc lập.
 - Ta có thể tạo các ứng dụng giao tiếp thông minh hơn chỉ bằng cách thay đổi View, hoặc cũng có thể thay thế Database bằng cách sử dụng Web service, ...



Hết